

```

// src/models/dedup/compressor.rs

use crate::models::dedup::tensor::DedupCountTensor;
use hashbrown::HashMap as AHashMap;
use crate::models::dedup::types::{UniqueTail, Sandbag};
use crate::models::dedup::avx512_kernel;
use crate::models::convert::common::{CompressOutput, CHUNK_SIZE};

impl DedupCountTensor {

    // Single entry point for compressing a tensor's weights.
    // Owns the full-precision escape, the >CHUNK_SIZE chunk split, and the
    // core/sandbag serialization. No other logic belongs here.
    pub fn compress_job(
        weights: &[f32],
        prefix_digits: usize,
        truncate_rounds: usize,
    ) -> CompressOutput {
        if Self::should_be_full_precision(weights) {
            let mut core = Vec::with_capacity(4 + weights.len() * 4);
            core.extend_from_slice(&1u32.to_le_bytes());
            for &w in weights {
                core.extend_from_slice(&w.to_le_bytes());
            }
            return CompressOutput {
                core,
                sandbag: Vec::new(),
                prefix_count: 0,
                unique_tail_count: 0,
                shared_weights: 0,
                mean_precision_lost: 0.0,
                full_precision: true,
            };
        }

        if weights.len() > CHUNK_SIZE {
            let n_chunks = (weights.len() + CHUNK_SIZE - 1) / CHUNK_SIZE;
            let mut core = Vec::new();
            core.extend_from_slice(&(n_chunks as u32).to_le_bytes());
            let mut sandbag = Vec::new();
            let mut prefix_count = 0;
            let mut unique_tail_count = 0;
            let mut shared_weights = 0;
            let mut mean_pl_sum = 0.0f32;
            let mut chunk_count = 0u32;

            for chunk in weights.chunks(CHUNK_SIZE) {
                let (t, m) = Self::compress(chunk, prefix_digits, truncate_rounds);
                core.extend(crate::models::convert::core::serialize_core(&t));
                sandbag.extend(m.to_bytes());
                prefix_count += t.prefixes.len();
                unique_tail_count += t.unique_tail_count();
                shared_weights += t.shared_tail_weights();
                mean_pl_sum += t.avg_precision_lost;
                chunk_count += 1;
            }

            return CompressOutput {

```

```

        core,
        sandbag,
        prefix_count,
        unique_tail_count,
        shared_weights,
        mean_precision_lost: mean_pl_sum / chunk_count.max(1) as f32,
        full_precision: false,
    };
} else {
    let (t, m) = Self::compress(weights, prefix_digits, truncate_rounds);
    let core = crate::models::convert::core::serialize_core(&t);
    let sandbag = m.to_bytes();
    let mut full_core = Vec::with_capacity(4 + core.len());
    full_core.extend_from_slice(&1u32.to_le_bytes());
    full_core.extend(core);

    CompressOutput {
        core: full_core,
        sandbag,
        prefix_count: t.prefixes.len(),
        unique_tail_count: t.unique_tail_count(),
        shared_weights: t.shared_tail_weights(),
        mean_precision_lost: t.avg_precision_lost,
        full_precision: false,
    }
}

pub fn should_be_full_precision(weights: &[f32]) -> bool {
    let step = (weights.len() / 1000).max(1);
    for i in (0..weights.len()).step_by(step) {
        if weights[i].abs() > 2.0 {
            return true;
        }
    }
    if weights.len() < 8192 {
        return true;
    }
    false
}

pub fn compress(weights: &[f32], prefix_digits: usize, truncate_rounds: usize)
-> (Self, Sandbag) {
    let initial_tail_digits = Self::TOTAL_DIGITS - prefix_digits;
    let prefix_scale = 10f32.powi(prefix_digits as i32);
    let tail_scale = 10f32.powi(Self::TOTAL_DIGITS as i32);
    let n = weights.len();

    // — Step 1: Group by prefix —
    let mut prefix_map: AHashMap<u16, u16> = AHashMap::with_capacity(512);
    let mut prefixes: Vec<u16> = Vec::new();
    let mut prefix_idx = vec![0u16; n];
    let mut sign_bits = vec![0u8; (n + 7) / 8];
    let mut group_tails: Vec<Vec<u32>> = Vec::new();

    for (i, &w) in weights.iter().enumerate() {
        let sign = w < 0.0;
        let abs_w = w.abs();
        let prefix_int = (abs_w * prefix_scale).floor() as u16;

```

```

let prefix_val = prefix_int as f32 / prefix_scale;
let tail_val = abs_w - prefix_val;
let tail_int = (tail_val * tail_scale).round() as u32;

let group_idx = match prefix_map.get(&prefix_int) {
    Some(&idx) => idx,
    None => {
        let idx = prefix_map.len() as u16;
        prefix_map.insert(prefix_int, idx);
        prefixes.push(prefix_int);
        group_tails.push(Vec::new());
        idx
    }
};

prefix_idx[i] = group_idx;
if sign { sign_bits[i / 8] |= 1 << (i % 8); }
group_tails[group_idx as usize].push(tail_int);
}

// — Step 2: Truncation with averaging (Zero-allocation optimization) —
let mut global_loss_sum = 0.0f32;
let mut global_loss_count = 0usize;
let mut current_tail_digits = initial_tail_digits;
let mut round_ups = vec![Vec::with_capacity(truncate_rounds);
prefixes.len()];

for _round in 0..truncate_rounds {
    let current_divisor = 10f32.powi((prefix_digits + current_tail_digits)
as i32);
    let next_divisor = 10f32.powi((prefix_digits + current_tail_digits - 1)
as i32);

    for (gidx, gt) in group_tails.iter_mut().enumerate() {
        if gt.is_empty() {
            round_ups[gidx].push(false);
            continue;
        }

        let mut sum_last_digits = 0u64;
        for &tail in gt.iter() {
            sum_last_digits += (tail % 10) as u64;
        }

        let avg = sum_last_digits as f32 / gt.len() as f32;
        let round_up = avg > 5.0;
        round_ups[gidx].push(round_up);

        for tail in gt.iter_mut() {
            let old_val = *tail as f32 / current_divisor;
            *tail = if round_up { *tail / 10 + 1 } else { *tail / 10 };
            let new_val = *tail as f32 / next_divisor;
            global_loss_sum += (old_val - new_val).abs();
            global_loss_count += 1;
        }
    }
    current_tail_digits -= 1;
}

```

```

let global_avg_lost = global_loss_sum / global_loss_count.max(1) as f32;

// — Step 3: Find unique tails + count —
let mut tail_counts: AHashMap<u16, u32> = AHashMap::new();
for gt in &group_tails {
    for &tail in gt {
        let tv = tail as u16;
        *tail_counts.entry(tv).or_insert(0) += 1;
    }
}

let mut unique_tail_values: Vec<u16> =
tail_counts.keys().copied().collect();
unique_tail_values.sort_unstable();

let tail_idx_map: AHashMap<u16, u16> = unique_tail_values.iter()
    .enumerate()
    .map(|(i, &v)| (v, i as u16))
    .collect();

let unique_tails: Vec<UniqueTail> = unique_tail_values.iter().map(|&v| {
    UniqueTail { value: v, repeat_count: tail_counts[&v] }
}).collect();

// — Step 4: Build tail_idx (AVX-512 with safe fallback) —
let mut tail_idx = vec![0u16; n];

if is_x86_feature_detected!("avx512f") && n % 16 == 0 {
    let mut out_tail_ints = vec![0u32; n];
    unsafe {
        avx512_kernel::avx512_reconstruct_tails(
            weights,
            prefix_scale,
            tail_scale,
            &prefix_idx,
            &round_ups,
            truncate_rounds,
            &mut out_tail_ints,
        );
    }
    for i in 0..n {
        let tv = out_tail_ints[i] as u16;
        tail_idx[i] = tail_idx_map.get(&tv).copied().unwrap_or(0);
    }
} else {
    // Scalar fallback path
    for i in 0..n {
        let abs_w = weights[i].abs();
        let prefix_int = (abs_w * prefix_scale).floor() as u16;
        let prefix_val = prefix_int as f32 / prefix_scale;
        let tail_val = abs_w - prefix_val;
        let mut tail_int = (tail_val * tail_scale).round() as u32;
        let gidx = prefix_idx[i] as usize;
        for round in 0..truncate_rounds {
            let ru = round_ups[gidx].get(round).copied().unwrap_or(false);
            tail_int = if ru { tail_int / 10 + 1 } else { tail_int / 10 };
        }
        let tv = tail_int as u16;
        tail_idx[i] = tail_idx_map.get(&tv).copied().unwrap_or(0);
    }
}

```

```

    }
}

let prefix_counts: Vec<u32> = (0..prefixes.len())
    .map(|gidx| group_tails[gidx].len() as u32)
    .collect();

let tensor = Self {
    prefixes, prefix_counts, unique_tails,
    count: n, prefix_digits,
    tail_digits: current_tail_digits,
    avg_precision_lost: global_avg_lost,
};

let tail_width: u8 = if truncate_rounds >= 3 { 0 } else { 1 };
let sandbag = Sandbag { prefix_idx, tail_idx, tail_width, sign_bits, count:
n };
(tensor, sandbag)
}

pub fn compress_from_gpu(
    weights: &[f32],
    prefix_bits: &[u32],
    tails: &[u32],
    signs: &[u32],
    prefix_digits: usize,
    truncate_rounds: usize,
) -> (Self, Sandbag) {
    let initial_tail_digits = Self::TOTAL_DIGITS - prefix_digits;
    let prefix_scale = 10f32.powi(prefix_digits as i32);
    let n = weights.len();

    let mut prefix_map: AHashMap<u16, u16> = AHashMap::with_capacity(512);
    let mut prefixes: Vec<u16> = Vec::new();
    let mut prefix_idx = vec![0u16; n];
    let mut sign_bits = vec![0u8; (n + 7) / 8];
    let mut group_tails: Vec<Vec<u32>> = Vec::new();

    for i in 0..n {
        let pv = f32::from_bits(prefix_bits[i]);
        let prefix_int = (pv * prefix_scale).floor() as u16;
        let group_idx = match prefix_map.get(&prefix_int) {
            Some(&idx) => idx,
            None => {
                let idx = prefix_map.len() as u16;
                prefix_map.insert(prefix_int, idx);
                prefixes.push(prefix_int);
                group_tails.push(Vec::new());
                idx
            }
        };

        prefix_idx[i] = group_idx;
        if signs[i] != 0 { sign_bits[i / 8] |= 1 << (i % 8); }
        group_tails[group_idx as usize].push(tails[i]);
    }

    let mut global_loss_sum = 0.0f32;
    let mut global_loss_count = 0usize;

```

```

        let mut current_tail_digits = initial_tail_digits;
        let mut round_ups = vec![Vec::with_capacity(truncate_rounds);
prefixes.len()];

        for _round in 0..truncate_rounds {
            let current_divisor = 10f32.powi((prefix_digits + current_tail_digits)
as i32);
            let next_divisor = 10f32.powi((prefix_digits + current_tail_digits - 1)
as i32);

            for (gidx, gt) in group_tails.iter_mut().enumerate() {
                if gt.is_empty() {
                    round_ups[gidx].push(false);
                    continue;
                }

                let mut sum_last_digits = 0u64;
                for &tail in gt.iter() {
                    sum_last_digits += (tail % 10) as u64;
                }
                let avg = sum_last_digits as f32 / gt.len() as f32;
                let round_up = avg > 5.0;
                round_ups[gidx].push(round_up);

                for tail in gt.iter_mut() {
                    let old_val = *tail as f32 / current_divisor;
                    *tail = if round_up { *tail / 10 + 1 } else { *tail / 10 };
                    let new_val = *tail as f32 / next_divisor;
                    global_loss_sum += (old_val - new_val).abs();
                    global_loss_count += 1;
                }
            }
            current_tail_digits -= 1;
        }

        let global_avg_lost = global_loss_sum / global_loss_count.max(1) as f32;

        // — Step 3: Find unique tails + count —
        let mut tail_counts: AHashMap<u16, u32> = AHashMap::new();
        for gt in &group_tails {
            for &tail in gt {
                let tv = tail as u16;
                *tail_counts.entry(tv).or_insert(0) += 1;
            }
        }

        let mut unique_tail_values: Vec<u16> =
tail_counts.keys().copied().collect();
        unique_tail_values.sort_unstable();

        let tail_idx_map: AHashMap<u16, u16> = unique_tail_values.iter()
.enumerate()
.map(|(i, &v)| (v, i as u16))
.collect();

        let unique_tails: Vec<UniqueTail> = unique_tail_values.iter().map(|&v| {
            UniqueTail { value: v, repeat_count: tail_counts[&v] }
        }).collect();

```

```

// — Step 4: Build tail_idx (AVX-512 with safe fallback) —
let mut tail_idx = vec![0u16; n];
    let tail_scale = 10f32.powi(Self::TOTAL_DIGITS as i32);

if is_x86_feature_detected!("avx512f") && n % 16 == 0 {
    let mut out_tail_ints = vec![0u32; n];
    unsafe {
        avx512_kernel::avx512_reconstruct_tails(
            weights,
            prefix_scale,
            tail_scale,
            &prefix_idx,
            &round_ups,
            truncate_rounds,
            &mut out_tail_ints,
        );
    }
    for i in 0..n {
        let tv = out_tail_ints[i] as u16;
        tail_idx[i] = tail_idx_map.get(&tv).copied().unwrap_or(0);
    }
} else {
    // Scalar fallback path
    for i in 0..n {
        let abs_w = weights[i].abs();
        let prefix_int = (abs_w * prefix_scale).floor() as u16;
        let prefix_val = prefix_int as f32 / prefix_scale;
        let tail_val = abs_w - prefix_val;
        let mut tail_int = (tail_val * tail_scale).round() as u32;
        let gidx = prefix_idx[i] as usize;
        for round in 0..truncate_rounds {
            let ru = round_ups[gidx].get(round).copied().unwrap_or(false);
            tail_int = if ru { tail_int / 10 + 1 } else { tail_int / 10 };
        }
        let tv = tail_int as u16;
        tail_idx[i] = tail_idx_map.get(&tv).copied().unwrap_or(0);
    }
}

let prefix_counts: Vec<u32> = (0..prefixes.len())
    .map(|gidx| group_tails[gidx].len() as u32)
    .collect();

let tensor = Self {
    prefixes, prefix_counts, unique_tails,
    count: n, prefix_digits,
    tail_digits: current_tail_digits,
    avg_precision_lost: global_avg_lost,
};

let tail_width: u8 = if truncate_rounds >= 3 { 0 } else { 1 };
let sandbag = Sandbag { prefix_idx, tail_idx, tail_width, sign_bits, count:
n };
    (tensor, sandbag)
}
}

// src/models/dedup/decompressor.rs

```

```

use crate::models::dedup::tensor::DedupCountTensor;
use hashbrown::HashMap as AHashMap;
use crate::models::dedup::types::{Sandbag, GlobalTable};

impl DedupCountTensor {

    pub fn decompress_all(&self, sandbag: &Sandbag) -> Vec<f32> {
        let prefix_scale = 10f32.powi(self.prefix_digits as i32);
        let divisor = 10f32.powi((self.prefix_digits + self.tail_digits) as i32);
        let mut result = Vec::with_capacity(self.count);

        for i in 0..self.count {
            let p_idx = sandbag.prefix_idx.get(i).copied().unwrap_or(0) as usize;
            let t_idx = sandbag.tail_idx.get(i).copied().unwrap_or(0) as usize;

            let prefix_int = self.prefixes.get(p_idx).copied().unwrap_or(0);
            let prefix = prefix_int as f32 / prefix_scale;
            let tail = self.unique_tails.get(t_idx).map(|ut|
ut.value).unwrap_or(0);

            let mut value = prefix + tail as f32 / divisor;
            value += self.avg_precision_lost;

            let sign = (sandbag.sign_bits.get(i / 8).copied().unwrap_or(0) >> (i %
8)) & 1 != 0;
            result.push(if sign { -value } else { value });
        }
        result
    }

    pub fn decompress_all_global(&self, sandbag: &Sandbag, global: &GlobalTable) ->
Vec<f32> {
        let tail_divisor = 10f32.powi((global.prefix_digits + self.tail_digits) as
i32);
        let gt_scale = 10f32.powi(global.prefix_digits as i32);
        let scale_diff = global.prefix_digits as i32 - self.prefix_digits as i32;

        let mut prefix_lookup: AHashMap<u16, usize> =
AHashMap::with_capacity(self.prefixes.len());
        for &pv in &self.prefixes {
            let norm = if scale_diff > 0 {
                (pv as u32) * 10u32.pow(scale_diff as u32)
            } else {
                pv as u32
            };
            if !prefix_lookup.contains_key(&pv) {
                if let Some(gi) = global.prefixes.iter().position(|&gp| (gp *
gt_scale).round() as u32 == norm) {
                    prefix_lookup.insert(pv, gi);
                }
            }
        }

        let mut tail_lookup: AHashMap<(u16, u16), usize> = AHashMap::new();
        for (cp_idx, &pv) in self.prefixes.iter().enumerate() {
            let gp_idx = *prefix_lookup.get(&pv).unwrap_or(&0);
            let global_tails = &global.tails_for_prefix[gp_idx];
            let mut val_to_gi: AHashMap<u32, usize> =
AHashMap::with_capacity(global_tails.len());

```



```

        for (gi, &t) in global_tails.iter().enumerate() {
            val_to_gi.insert(t, gi);
        }
        for (ct_idx, ut) in self.unique_tails.iter().enumerate() {
            if let Some(&gi) = val_to_gi.get(&(ut.value as u32)) {
                tail_lookup.insert((cp_idx as u16, ct_idx as u16), gi);
            }
        }
    }

    let mut result = Vec::with_capacity(self.count);
    for i in 0..self.count {
        let p_idx = sandbag.prefix_idx.get(i).copied().unwrap_or(0);
        let t_idx = sandbag.tail_idx.get(i).copied().unwrap_or(0) as u16;

        let pv = self.prefixes.get(p_idx as usize).copied().unwrap_or(0);
        let gp = *prefix_lookup.get(&pv).unwrap_or(&0);
        let gt = *tail_lookup.get(&(p_idx, t_idx)).unwrap_or(&0);

        let prefix = global.prefixes[gp];
        let tail = global.tails_for_prefix[gp][gt];

        let mut value = prefix + tail as f32 / tail_divisor;
        value += self.avg_precision_lost;

        let sign = (sandbag.sign_bits.get(i / 8).copied().unwrap_or(0) >> (i %
8)) & 1 != 0;
        result.push(if sign { -value } else { value });
    }
    result
}
}

```

// src/models/dedup/global\_lookup.rs

```

use crate::models::dedup::types::{GlobalTable, Sandbag, ChunkRemap};
use crate::models::dedup::tensor::DedupCountTensor;

```

```

impl GlobalTable {
    /// Constructs a unified GlobalTable from a slice of individual tensor chunks.
    pub fn new(chunks: &[DedupCountTensor]) -> Self {
        if chunks.is_empty() {
            return Self {
                prefix_digits: 0,
                prefixes: Vec::new(),
                tails_for_prefix: Vec::new(),
            };
        }

        // 1. Pull the unified scale config from the first chunk block
        let prefix_digits = chunks[0].prefix_digits;
        let prefix_scale = 10f32.powi(prefix_digits as i32);

        // 2. Collect and deduplicate all unique integer prefixes across chunks
        let mut unique_prefixes: Vec<u16> = Vec::new();
        for tensor in chunks {
            for &p in &tensor.prefixes {
                if !unique_prefixes.contains(&p) {

```

```

        unique_prefixes.push(p);
    }
}
unique_prefixes.sort_unstable();

// 3. Map integer prefixes to the raw f32 scale values your struct fields
expect
let mut prefixes_f32 = Vec::with_capacity(unique_prefixes.len());
let mut tails_for_prefix = vec![Vec::new(); unique_prefixes.len()];

for &p in &unique_prefixes {
    prefixes_f32.push(p as f32 / prefix_scale);
}

// 4. Group unique tail values under their respective global prefix index
lanes
for tensor in chunks {
    for &p in &tensor.prefixes {
        if let Ok(global_p_idx) = unique_prefixes.binary_search(&p) {
            let target_prefix_tails = &mut tails_for_prefix[global_p_idx];

            for ut in &tensor.unique_tails {
                let t_val = ut.value as u32;
                if !target_prefix_tails.contains(&t_val) {
                    target_prefix_tails.push(t_val);
                }
            }
        }
    }
}

// 5. Return your exact struct fields populated cleanly
Self {
    prefix_digits,
    prefixes: prefixes_f32,
    tails_for_prefix,
}

}

/// Generates a local chunk remapping descriptor table.
pub fn build_chunk_remap(&self, tensor: &DedupCountTensor) -> ChunkRemap {
    let mut global_tail_indices =
Vec::with_capacity(tensor.unique_tails.len());
    let gt_scale = 10f32.powi(self.prefix_digits as i32);
    let scale_diff = self.prefix_digits as i32 - tensor.prefix_digits as i32;

    for ut in &tensor.unique_tails {
        let mut matched_global_idx = 0u16;
        'outer: for &pv in &tensor.prefixes {
            let norm = if scale_diff > 0 {
                (pv as u32) * 10u32.pow(scale_diff as u32)
            } else {
                pv as u32
            };
            if let Some(gp_idx) = self.prefixes.iter().position(|&gp| (gp *
gt_scale).round() as u32 == norm) {
                if let Some(pos) =
self.tails_for_prefix[gp_idx].iter().position(|&t| t == ut.value as u32) {

```

```

        matched_global_idx = pos as u16;
        break 'outer;
    }
}
}
global_tail_indices.push(matched_global_idx);
}
ChunkRemap { global_tail_indices }
}

/// Decompresses elements safely using a chunk remap cache configuration.
pub fn decompress_with_remap(&self, sandbag: &Sandbag, tensor:
&DedupCountTensor, remap: Option<ChunkRemap>) -> Vec<f32> {
    if let Some(m) = remap {
        let divisor = 10f32.powi((self.prefix_digits + tensor.tail_digits) as
i32);
        let mut result = Vec::with_capacity(sandbag.count);
        let gt_scale = 10f32.powi(self.prefix_digits as i32);
        let scale_diff = self.prefix_digits as i32 - tensor.prefix_digits as
i32;

        for i in 0..sandbag.count {
            let p_idx = sandbag.prefix_idx[i] as usize;
            let t_idx = sandbag.tail_idx[i] as usize;

            let pv = tensor.prefixes[p_idx];
            let norm = if scale_diff > 0 { (pv as u32) * 10u32.pow(scale_diff
as u32) } else { pv as u32 };

            let gp = self.prefixes.iter().position(|&gp| (gp *
gt_scale).round() as u32 == norm).unwrap_or(0);
            let local_mapped_tail_idx =
m.global_tail_indices.get(t_idx).copied().unwrap_or(0) as usize;
            let tail =
self.tails_for_prefix[gp].get(local_mapped_tail_idx).copied().unwrap_or(0);

            let mut value = self.prefixes[gp] + tail as f32 / divisor;
            value += tensor.avg_precision_lost;

            let sign = (sandbag.sign_bits[i / 8] >> (i % 8)) & 1 != 0;
            result.push(if sign { -value } else { value });
        }
    } else {
        tensor.decompress_all(sandbag)
    }
}
}

```

// src/models/dedup/gpu\_helpers.rs

// src/models/dedup/mod.rs

```

pub mod types;
pub mod tensor;
pub mod compressor;
pub mod truncation;

```

```

pub use super::avx512_kernel;
pub mod serialization;
pub mod decompressor;
pub mod global_lookup;
pub mod gpu_helpers;

// src/models/dedup/serialization.rs

use crate::models::dedup::types::Sandbag;

impl Sandbag {
    /// Serializes the sandbag structure into a packed binary payload.
    pub fn to_bytes(&self) -> Vec<u8> {
        let mut bytes = Vec::new();
        bytes.extend_from_slice(&(self.count as u32).to_le_bytes());
        bytes.push(self.tail_width);

        // Write out prefix index arrays
        bytes.extend_from_slice(&(self.prefix_idx.len() as u32).to_le_bytes());
        for &p in &self.prefix_idx {
            bytes.extend_from_slice(&p.to_le_bytes());
        }

        // Write out tail index arrays
        bytes.extend_from_slice(&(self.tail_idx.len() as u32).to_le_bytes());
        for &t in &self.tail_idx {
            bytes.extend_from_slice(&t.to_le_bytes());
        }

        // Append bitpacked sign streams
        bytes.extend_from_slice(&(self.sign_bits.len() as u32).to_le_bytes());
        bytes.extend_from_slice(&self.sign_bits);

        bytes
    }

    /// Deserializes a packed sandbag instance from a byte array slice.
    pub fn from_bytes(data: &[u8]) -> Option<Self> {
        if data.len() < 9 { return None; }
        let mut pos = 0;

        let count = u32::from_le_bytes(data[pos..pos+4].try_into().ok()?) as usize;
        pos += 4;
        let tail_width = data[pos];
        pos += 1;

        let p_len = u32::from_le_bytes(data[pos..pos+4].try_into().ok()?) as usize;
        pos += 4;
        if data.len() < pos + (p_len * 2) { return None; }
        let mut prefix_idx = Vec::with_capacity(p_len);
        for _ in 0..p_len {
            prefix_idx.push(u16::from_le_bytes(data[pos..pos+2].try_into().ok()?));
            pos += 2;
        }

        let t_len = u32::from_le_bytes(data[pos..pos+4].try_into().ok()?) as usize;
        pos += 4;
        if data.len() < pos + (t_len * 2) { return None; }
    }
}

```

```

        let mut tail_idx = Vec::with_capacity(t_len);
        for _ in 0..t_len {
            tail_idx.push(u16::from_le_bytes(data[pos..pos+2].try_into().ok()?));
            pos += 2;
        }

        let s_len = u32::from_le_bytes(data[pos..pos+4].try_into().ok()?) as usize;
        pos += 4;
        if data.len() < pos + s_len { return None; }
        let sign_bits = data[pos..pos+s_len].to_vec();

        Some(Self {
            prefix_idx,
            tail_idx,
            tail_width,
            sign_bits,
            count,
        })
    }

    /// Resolves the true total byte size of this serialized segment.
    pub fn bytes(&self) -> usize {
        // count(4) + width(1) + structures lengths headers + payloads
        4 + 1 + 4 + (self.prefix_idx.len() * 2) + 4 + (self.tail_idx.len() * 2) + 4
    }
    + self.sign_bits.len()
}

// src/models/dedup/tensor.rs

use crate::models::dedup::types::UniqueTail;

#[derive(Clone, Debug)]
pub struct DedupCountTensor {
    pub prefixes: Vec<u16>,
    pub prefix_counts: Vec<u32>,
    pub unique_tails: Vec<UniqueTail>,
    pub count: usize,
    pub prefix_digits: usize,
    pub tail_digits: usize,
    pub avg_precision_lost: f32,
}

impl DedupCountTensor {
    pub const TOTAL_DIGITS: usize = 7;

    pub fn unique_tail_count(&self) -> usize {
        self.unique_tails.len()
    }

    pub fn shared_tail_weights(&self) -> usize {
        self.unique_tails.iter()
            .filter(|ut| ut.repeat_count > 1)
            .map(|ut| ut.repeat_count as usize)
            .sum()
    }

    pub fn compressed_bytes(&self) -> usize {

```

```

        let header = 4 + 4 + 4 + 4 + 4;
        let front = self.prefixes.len() * 2 + self.unique_tails.len() * 2;
        let flags = 3;
        header + front + flags
    }
}

```

```
// src/models/dedup/truncation.rs
```

```
// src/models/dedup/types.rs
```

```

#[derive(Clone, Copy, Debug)]
pub enum DataFlag {
    GapFlag = 0xFD,
    TailFlag = 0xFE,
    CountFlag = 0xFF,
}

#[derive(Clone, Debug, PartialEq)]
pub struct UniqueTail {
    pub value: u16,
    pub repeat_count: u32,
}

#[derive(Clone, Debug)]
pub struct Sandbag {
    pub prefix_idx: Vec<u16>,
    pub tail_idx: Vec<u16>,
    pub tail_width: u8,
    pub sign_bits: Vec<u8>,
    pub count: usize,
}

#[derive(Clone, Debug)]
pub struct GlobalTable {
    pub prefix_digits: usize,
    pub prefixes: Vec<f32>,
    pub tails_for_prefix: Vec<Vec<u32>>,
}

#[derive(Clone, Debug)]
pub struct ChunkRemap {
    pub global_tail_indices: Vec<u16>,
}

```